

A machine-checked zero-knowledge argument of knowledge for a machine-checked formalisation

Formal development accompanying the Lean 4 formalisation of
M.-c. Kang, *Frobenius groups and retract rationality* (arXiv:1204.1796)

Abstract

We describe, and prove correct in Lean 4, a zero-knowledge argument of knowledge whose statement is about the formal development that carries the proof. The development is a Lean 4 formalisation of Kang’s paper *Frobenius groups and retract rationality*. We arithmetise two things into a single rank-1 constraint system over the prime field $\mathbb{F}_q$, $q = 2^{31} - 1$: (i) an algebraic hash chain whose preimage is the field encoding of the SHA-256 digest of the mathematical core of the development — the “code” — and (ii) the order formula $|\mathrm{SL}_2(\mathbb{F}_p)| = p(p^2 - 1)$ evaluated at the four primes $p = 3, 5, 7, 11$ that occur as the characteristics of the Frobenius complements in the paper, whose output wires are proved equal to the group orders established by the formalisation itself. We commit to the wire assignment with Pedersen commitments and prove each constraint with the multiplication Σ -protocol. Completeness, knowledge soundness (via an explicit extractor) and perfect honest-verifier zero knowledge are all theorems in Lean, as are the soundness and completeness of the arithmetisation itself. Every statement in this note is backed by a Lean declaration that compiles with no `sorry` and no axioms beyond `propepxt`, `Classical.choice` and `Quot.sound`.

1 What is being claimed

The underlying object is a Lean 4 formalisation of M.-c. Kang, *Frobenius groups and retract rationality*: Frobenius groups and complements, the *GZ*-condition, Burnside’s classification of 2-groups with a unique involution, the Zassenhaus types, and the retract-rationality reductions of the paper’s §4, together with a “complexity lattice” recording the exact orders of the groups that occur (Q_8 , $\mathrm{SL}_2(\mathbb{F}_3)$, the binary octahedral group, A_5 , $\mathrm{SL}_2(\mathbb{F}_5)$, $\hat{S}_5$, $\mathrm{SL}_2(\mathbb{F}_7)$, $\mathrm{SL}_2(\mathbb{F}_{11})$, ...).

The claim we make in zero knowledge is the conjunction of two things:

1. **I know the code.** I know four field elements $x_0, \dots, x_3$ — the four 64-bit limbs of the SHA-256 digest of the mathematical core of the development, reduced modulo q — whose algebraic hash is the published digest

$$d = 485918357 \in \mathbb{F}_q, \quad q = 2^{31} - 1.$$

2. **The mathematics checks out.** The same assignment carries, on four designated output wires, the values 24, 120, 336, 1320 obtained *inside the circuit* from the order formula $p(p^2 - 1)$ at $p = 3, 5, 7, 11$; and these four numbers are exactly the orders of $\mathrm{SL}_2(\mathbb{F}_3)$, $\mathrm{SL}_2(\mathbb{F}_5)$, $\mathrm{SL}_2(\mathbb{F}_7)$, $\mathrm{SL}_2(\mathbb{F}_{11})$ as proved in the formalisation.

Neither the transcript of the protocol nor the commitments reveal $x_0, \dots, x_3$; this is a theorem (§5.3), not a slogan.

Remark 1.1. The repository is public, so the limbs are also present in it: they must be, if the argument is to be run end to end and re-checked by a reader. What is proved here is that the *protocol* does not reveal them — the verifier’s entire view is reproduced by a simulator that never sees the assignment. A deployment in which the source is genuinely secret uses the same theorems verbatim.

2 Circuits: rank-1 constraint systems

Definition 2.1 (Kang1204.ZK.R1CS). For a commutative ring F and $m, n \in \mathbb{N}$, an *R1CS* is a triple of matrices $A, B, C: \text{Fin } m \rightarrow \text{Fin } n \rightarrow F$. An assignment $z: \text{Fin } n \rightarrow F$ *satisfies* it when

$$\langle A_i, z \rangle \cdot \langle B_i, z \rangle = \langle C_i, z \rangle \quad \text{for every } i.$$

The dimensions are part of the type and wire indices are elements of $\text{Fin } n$: a circuit cannot mention a wire it does not have, and a witness of the wrong width is not a term of the right type. This is what “the types are working” means here, and it is enforced by the kernel rather than by a side condition. Conjunction of circuits is `R1CS.append`, with `R1CS.satisfies_append` the corresponding logical equivalence.

3 Commitments

Let q be prime and let G be an abelian group which is a $\mathbb{Z}/q$ -module (a group of exponent q , written additively; in multiplicative notation $m \cdot g + r \cdot h$ reads $g^m h^r$). Fix bases $g, h \in G$.

Definition 3.1 (`Params.com`). $\text{com}(m, r) = m \cdot g + r \cdot h$.

Theorem 3.2 (`Homomorphy, Params.com_linearCombination`). $\sum_j u_j \cdot \text{com}(z_j, \rho_j) = \text{com}(\sum_j u_j z_j, \sum_j u_j \rho_j)$

Consequently *linear* constraints cost nothing: the verifier forms the required linear combinations of the public commitments itself.

Theorem 3.3 (`Binding, Params.dlog_of_two_openings`). If $\text{com}(m, r) = \text{com}(m', r')$ with $m \neq m'$ then $g = \frac{r' - r}{m - m'} \cdot h$.

This is a constructive reduction: an adversary producing two openings *computes* the discrete logarithm of g in base h . The usable form of the hypothesis is

$$\text{Independent}(g, h) : \equiv \forall a, b, a \cdot g + b \cdot h = 0 \implies a = 0 \wedge b = 0,$$

under which openings are unique (`Params.opening_unique`).

Theorem 3.4 (`Perfect hiding, Params.com_hidingEquiv`). If $g = \beta \cdot h$ then for all messages m, m' the map $r \mapsto r + (m - m')\beta$ is a bijection of the randomness space with $\text{com}(m, r) = \text{com}(m', r + (m - m')\beta)$.

A bijection transports the uniform distribution to the uniform distribution, so the distribution of a commitment is independent of the committed value.

Remark 3.5. No pair of bases in a group satisfies both hypotheses unconditionally; that is the usual binding/hiding trade-off. Both are shown satisfiable separately in `RequestProject/ZK/Instances.lean` (the plane $\mathbb{F}_q^2$ with the coordinate axes for binding, the line $\mathbb{F}_q$ for hiding), so no statement below is vacuous.

4 Σ -protocols

Definition 4.1 (Kang1204.ZK.SigmaProtocol). A three-move public-coin protocol for a relation rel consists of commit, respond, verify. It is *complete* if an honest prover holding a witness is always accepted; *specialy sound* if there is an extractor turning two accepting transcripts with the same first message and different challenges into a witness; and *perfectly HVZK* if there is a simulator, seeing only the statement and the challenge, together with a bijection of the randomness space matching real and simulated transcripts.

Formalising zero knowledge by an explicit bijection avoids any measure theory: identically distributed is literally “equal up to a bijective renaming of the random tape”. From it we derive witness indistinguishability (`PerfectHVZK.witness_indistinguishable`): for two witnesses of the same statement the transcripts agree under a bijection of the randomness.

Two protocols are formalised.

Theorem 4.2 (Opening protocol; `openingComplete`, `openingSpecialSoundness`, `openingPerfectHVZK`). *For $x = \text{com}(m, r)$: send $a = \text{com}(s, t)$; receive e ; answer $(s + em, t + er)$; check $\text{com}(z_1, z_2) = a + e \cdot x$. This is complete, specially sound and perfectly HVZK.*

Theorem 4.3 (Multiplication protocol; `productComplete`, `productSpecialSoundness`, `productPerfectHVZK`). *For a statement (X, Y, Z) and relation $X = \text{com}(a, r_a) \wedge Z = a \cdot Y + \tau \cdot h$: send $M_1 = \text{com}(x, s_1)$, $M_2 = x \cdot Y + s_2 \cdot h$; receive e ; answer $(x + ea, s_1 + er_a, s_2 + e\tau)$; check $\text{com}(z, z_1) = M_1 + e \cdot X$ and $z \cdot Y + z_2 \cdot h = M_2 + e \cdot Z$. This is complete, specially sound and perfectly HVZK.*

Lemma 4.4 (`productProtocol_rel_product`). *If $Y = \text{com}(b, r_b)$, the relation above says exactly that $Z = \text{com}(ab, ar_b + \tau)$: the protocol proves a multiplicative relation between committed values.*

5 The argument for an arbitrary circuit

Commit to the assignment wire by wire, $C_j = \text{com}(z_j, \rho_j)$. For each constraint i the verifier forms

$$X_i = \sum_j A_{ij} \cdot C_j, \quad Y_i = \sum_j B_{ij} \cdot C_j, \quad Z_i = \sum_j C_{ij} \cdot C_j$$

out of public data, and the prover runs the multiplication protocol on (X_i, Y_i, Z_i) .

Theorem 5.1 (Completeness, `verify_of_satisfies`). *If z satisfies the circuit then, for every constraint, every prover randomness and every challenge, the verifier accepts.*

Theorem 5.2 (Knowledge soundness, `knowledge_soundness`). *Assume `Independent(g, h)` and $C_j = \text{com}(z_j, \rho_j)$. If for every constraint the prover produces two accepting transcripts with the same first message and distinct challenges, then z satisfies the circuit.*

The proof is the composite of the extractor of the multiplication protocol with `satisfies_of_constraintRe` binding turns the extracted witnesses into the equalities $\langle A_i, z \rangle \langle B_i, z \rangle = \langle C_i, z \rangle$.

Theorem 5.3 (Zero knowledge, `zero_knowledge_transcript`, `zero_knowledge_indistinguishable`). *Every transcript is produced, accepted, by a simulator that sees only the public statement and the challenge; and for any two satisfying assignments committed to the same group elements the transcripts coincide under a bijection of the prover’s randomness. Together with hiding of the commitments (`wire_commitment_hiding`) the verifier’s entire view is independent of the assignment.*

6 The self-referential circuit

6.1 Field, hash, digest

We work over $\mathbb{F}_q$ with $q = 2^{31} - 1$ (a Mersenne prime; the primality is checked in Lean). The hash is a MiMC-style chain

$$\text{acc}_0 = (0 + x_0)^3, \quad \text{acc}_i = (\text{acc}_{i-1} + x_i)^3 \quad (i = 1, 2, 3),$$

each round costing one squaring and one multiplication, i.e. two rank-1 constraints. The inputs $x_0, \dots, x_3$ are the four 64-bit limbs of the SHA-256 digest of all files of `RequestProject/` other

than `Main.lean` and the zero-knowledge layer, reduced modulo q ; the script `scripts/source_digest.sh` recomputes them. The published value is

$$d = \text{mimcHash}(x_0, x_1, x_2, x_3) = 485918357,$$

a Lean theorem (`sourceDigest_eq`), proved by kernel computation.

6.2 Wire layout

wire	content
0	the constant 1
1–4	$x_0, \dots, x_3$: the secret encoding of the source
5, 7, 9, 11	the squares $(\text{acc}_{i-1} + x_i)^2$
6, 8, 10, 12	the accumulators $\text{acc}_0, \dots, \text{acc}_3$
13–16	the primes 3, 5, 7, 11
17–20	their squares
21–24	the group orders 24, 120, 336, 1320

6.3 Constraints

index	constraint	meaning
0, 2, 4, 6	$(\text{acc}_{i-1} + x_i)^2 = t_i$	squaring in round i
1, 3, 5, 7	$t_i \cdot (\text{acc}_{i-1} + x_i) = \text{acc}_i$	cubing in round i
8	$(\text{acc}_3 - d) \cdot 1 = 0$	the accumulator is the public digest
9, 11, 13, 15	$p \cdot p = s$	squaring the prime
10, 12, 14, 16	$(s - 1) \cdot p = c$	the order formula $p(p^2 - 1)$
17–20	$(p - \bar{p}) \cdot 1 = 0$	the primes are the public 3, 5, 7, 11

So the circuit has $m = 21$ constraints on $n = 25$ wires: in Lean it is a term of type `R1CS Fq 21 25`.

6.4 The two directions

Theorem 6.1 (`selfCircuit_satisfies`). *The honest assignment satisfies all 21 constraints, at the digest which is the hash of its own input wires. In particular the published circuit is satisfiable (`publishedCircuit_sat`).*

Theorem 6.2 (`selfCircuit_sound`). *Conversely, let z be any assignment satisfying the circuit at digest d , with $z_0 = 1$. Then*

$$\text{mimcHash}(z_1, z_2, z_3, z_4) = d, \quad z_{21} = 24, \quad z_{22} = 120, \quad z_{23} = 336, \quad z_{24} = 1320.$$

Theorem 6.3 (`selfCircuit_orders_eq_card`). *The four output wires of the honest assignment are $|\text{SL}_2(\mathbb{F}_3)|$, $|\text{SL}_2(\mathbb{F}_5)|$, $|\text{SL}_2(\mathbb{F}_7)|$, $|\text{SL}_2(\mathbb{F}_{11})|$ as proved in the formalisation itself.*

The last theorem is the bridge between the circuit and the mathematics: the numbers the circuit computes are not decorative constants but the cardinalities that the group-theoretic part of the development establishes (by explicit computation for $\text{SL}_2(\mathbb{F}_3)$, $\text{SL}_2(\mathbb{F}_5)$, $\text{SL}_2(\mathbb{F}_7)$, $\text{SL}_2(\mathbb{F}_{11})$).

7 The self argument

Instantiating the general argument at `publishedCircuit` gives the main statement.

Theorem 7.1 (`self_zk_argument_of_knowledge`). *Let (g, h) be independent Pedersen bases and ρ any commitment randomness. Then:*

1. *the published circuit is satisfiable;*
2. *the honest prover, committing to the assignment built from the source limbs, is accepted on every constraint and every challenge;*
3. *if the wire commitments open to z , the constant wire is 1, and some prover answers two distinct challenges on every constraint with the same first messages, then $\text{mimcHash}(z_1, \dots, z_4) = 485918357$ and $z_{21}, \dots, z_{24} = 24, 120, 336, 1320$;*
4. *every transcript is produced, and accepted, by a simulator which sees only the public statement and the challenge.*

Item (3) is the argument of knowledge: a prover who convinces the verifier knows a preimage of the published digest — the code — and that same knowledge certifies the arithmetic facts. Item (4), with hiding of the commitments and witness indistinguishability, is the “without revealing it” part. Item (2) instantiated at concrete parameters is `planeParams_self_zk_argument_of_knowled`.

8 Scope, and what is *not* claimed

- **Binding is conditional, and honestly so.** Unconditional binding is impossible against an unbounded adversary; what is proved is the reduction (two openings give a discrete logarithm) and the consequences of the hypothesis `Independent( $g, h$ )`. The choice of group is a deployment matter.
- **Honest-verifier zero knowledge.** The simulator is for the honest-verifier setting, as usual for Σ -protocols; the standard transformations to full zero knowledge, and the Fiat–Shamir transform to a non-interactive proof, are not formalised here.
- **Soundness error.** A single run has the usual soundness error $1/|\mathbb{F}_q|$ per constraint against a cheating prover that cannot answer two challenges; the theorem formalised is the extraction statement, not a probability bound.
- **The hash.** The MiMC-style chain over $\mathbb{F}_q$ with four rounds is a circuit-friendly algebraic hash; the theorems proved about it are exact algebraic statements (the circuit computes it, and the circuit’s satisfaction forces a preimage). Collision resistance is a cryptographic assumption, not a theorem, and $q = 2^{31} - 1$ is a demonstration parameter.
- **The digest covers the mathematics, not this note.** It is the SHA-256 of the group-theoretic core of the development; the zero-knowledge layer and `Main.lean` are excluded, since a digest cannot include the file that states it.

9 Reproducing

```
lake build          # the whole development, no sorry
./scripts/source_digest.sh  # recompute the limbs and the SHA-256
```

Everything is computable, and `RequestProject/ZK/Demo.lean` runs the protocol: it commits to all 25 wires, produces a first message, a challenge and a response for each of the 21 constraints, and evaluates the verifier, which returns `true` on every constraint. That the run is accepted is also a theorem, `demoAccepts`, obtained from completeness — so it holds for every randomness and every challenge, not only for the evaluated one. The same file exhibits, over the hiding parameters, a single group element opened to two different messages, and a Boolean verifier `verifyB` proved equivalent to the propositional one.

The zero-knowledge layer is `RequestProject/ZK/`: `Circuit.lean` (R1CS), `Pedersen.lean` (commitments), `Sigma.lean` (Σ -protocols), `OpeningProof.lean`, `ProductProof.lean`, `Argument.lean` (the argument for an arbitrary circuit), `SelfCircuit.lean` (the circuit and its two correctness theorems), `SelfZK.lean` (the instantiation) and `Instances.lean` (concrete parameters). Every theorem cited above compiles with no `sorry`, and `#print axioms` on each of them returns only `propext`, `Classical.choice`, `Quot.sound`.